

Guida allo Sviluppo di App Fiori Elements su SAP BTP ABAP Environment

Dalla creazione del progetto al deploy su tenant consumer

List Report + Object Page con OData V4

Archiva Group - Aprile 2026

Basato sull'esperienza del progetto Configurazione Conservazione

Indice

1. Prerequisiti e architettura
2. Struttura del progetto frontend
3. Creazione dei file dell'app
4. Configurazione del mockserver
5. Preview locale in BAS
6. Service Binding OData V4 (Web API)
7. Deploy della BSP
8. Configurazione IAM e Business Catalog
9. Configurazione Tile nel Launchpad
10. Communication Scenario e Arrangement
11. Deploy su tenant consumer
12. Multitenancy e isolamento dati
13. Lezioni apprese e best practice

1. Prerequisiti e architettura

Questa guida descrive il processo completo per creare, deployare e configurare un'applicazione SAP Fiori Elements (List Report + Object Page) su SAP BTP ABAP Environment, dal codice sorgente fino al tile nel Launchpad sui tenant consumer.

Prerequisiti

- SAP Business Application Studio (BAS) con dev space configurato
- Accesso al sistema ABAP BTP (ADT/Eclipse collegato)
- Repository Git (GitHub o altro) per il versioning
- Destination SAP BTP configurata (es. ABAP_Employee_RAP) con WebIDEUsage: odata_gen
- Package ABAP per gli oggetti (es. ZEDOC_MONITOR)

Architettura

L'app Fiori Elements si basa su template standard SAP (sap.fe.templates) che generano automaticamente la UI a partire dalle annotazioni OData. Il flusso e':

Layer	Componente	Descrizione
Backend ABAP	CDS View + Service Def + Service Binding	Modello dati e servizio OData V4
Frontend UI5	manifest.json + Component.js + annotation.xml	App Fiori Elements con annotazioni
Deploy	BSP (UI5 Repository)	App deployata come BSP sul sistema ABAP
Launchpad	IAM App + Business Catalog + Tile	Navigazione e autorizzazioni

2. Struttura del progetto frontend

La struttura di cartelle per un'app Fiori Elements standard:

```
nome_progetto/
+-- package.json
+-- ui5.yaml           # Proxy backend BTP
+-- ui5-deploy.yaml    # Configurazione deploy BSP
+-- ui5-mock.yaml      # Configurazione mockserver
+-- webapp/
    +-- index.html      # Bootstrap UI5
    +-- Component.js     # AppComponent Fiori Elements
    +-- manifest.json    # Configurazione app e routing
    +-- annotations/
        | +-- annotation.xml # Annotazioni UI
    +-- localService/
        | +-- mainService/
        |     +-- metadata.xml # Mock OData V4 metadata
        |     +-- *.json      # Mockdata
    +-- i18n/
        | +-- i18n.properties # Testi internazionalizzati
    +-- css/
        +-- style.css      # CSS custom
```

3. Creazione dei file dell'app

3.1 package.json

Definisce le dipendenze e gli script di build/deploy:

```
{
  "name": "nome-app",
  "version": "0.0.1",
  "scripts": {
    "start": "fiori run --open index.html",
    "start-mock": "fiori run --config ./ui5-mock.yaml --open index.html",
    "build": "ui5 build --clean-dest",
    "deploy": "fiori add deploy-config && npm run build && fiori deploy"
  },
  "devDependencies": {
    "@sap/ux-ui5-tooling": "1",
    "@ui5/cli": "^3.0.0",
    "@sap-ux/ui5-middleware-fe-mockserver": "^2"
  }
}
```

3.2 Component.js

Obbligatorio per Fiori Elements. Estende sap/fe/core/AppComponent:

```
sap.ui.define(
  ["sap/fe/core/AppComponent"],
  function (Component) {
    "use strict";
    return Component.extend("namespace.Component", {
      metadata: { manifest: "json" }
    });
  }
);
```

3.3 index.html

Bootstrap UI5. Usare il path relativo resources/sap-ui-core.js per il preview locale:

```
<script id="sap-ui-bootstrap"
  src="resources/sap-ui-core.js"
  data-sap-ui-theme="sap_horizon"
  data-sap-ui-resourceroots='{ "namespace": "." }'
  data-sap-ui-async="true"
  data-sap-ui-oninit="module:sap/ui/core/ComponentSupport">
</script>
```

Nota: NON usare il CDN (ui5.sap.com) per il bootstrap — causa pagina bianca in BAS.

3.4 manifest.json (elementi chiave)

Punti critici nel manifest:

- **dataSource URI:** deve puntare al Service Binding **V4 Web API** (non UI)
- **odataVersion:** "4.0"
- **routing targets:** usare sap.fe.templates.ListReport e sap.fe.templates.ObjectPage

- **flexEnabled:** true
- **sourceTemplate.id:** @sap/generator-fiori:list-report-object-page
- **sap.fe.templates** nelle dependencies libs

***Nota:** Per app multi-entita' (tab), usare la configurazione 'views.paths' nel target ListReport con SelectionVariant qualifier per ogni entita'.*

3.5 annotation.xml

Le annotazioni UI definiscono il layout. Regola fondamentale:

***Nota:** Il binding i18n ({{i18n>key}}) NON funziona nei file annotation.xml esterni. Usare SEMPRE testi diretti hardcoded (es. String="Codice Societa").*

Annotazioni obbligatorie:

Annotazione	Scopo
UI.HeaderInfo	Titolo e sottotitolo dell'Object Page
UI.SelectionFields	Filtri nella barra di ricerca
UI.LineItem	Colonne della tabella nella List Report
UI.DataPoint	Criticality (pallini colorati verde/rosso/grigio)
UI.Facets	Sezioni dell'Object Page
UI.FieldGroup	Campi dentro ogni sezione
Common.ValueList	Value help per i filtri
Core.Immutable	Campo non modificabile in edit mode

4. Configurazione del mockserver

Il file `ui5-mock.yaml` configura il mockserver per il preview locale:

```
specVersion: "3.0"
metadata:
  name: nomeapp
type: application
framework:
  name: SAPUI5
  version: "1.120.0"      # DEVE essere semver (x.y.z)
  libraries:
    - name: sap.m
    - name: sap.ui.core
    - name: sap.fe.core
    - name: sap.fe.templates
    - name: sap.ui.fl
server:
  customMiddleware:
    - name: sap-fe-mockserver
      afterMiddleware: compression
      configuration:
        services:
          - urlPath: /sap/opu/odata4/.../
            metadataPath: ../webapp/localService/mainService/metadata.xml
            mockdataPath: ../webapp/localService/mainService/
            generateMockData: false
        annotations:
          - urlPath: /annotations/annotation.xml
            localPath: ../webapp/annotations/annotation.xml
```

Errori comuni da evitare:

- Versione UI5 "1.120" non valida - usare "1.120.0" (formato semver)
- Middleware `beforeMiddleware: csp` non funziona - usare `afterMiddleware: compression`
- Proprieta' `services` deve essere un array (non service singolare)
- Il `urlPath` deve corrispondere esattamente all'URI nel `manifest.json`

5. Preview locale in BAS

Comandi per avviare il preview con mockdata in BAS:

```
cd /home/user/nome-progetto  
npm install  
npx ui5 serve --config ./ui5-mock.yaml --open index.html
```

Nota: Il tema `sap_horizon` non si carica completamente nel preview locale con `ui5 serve v3` (CSS mancante). Questo e' normale - il tema funziona correttamente dopo il deploy sulla BSP.

6. Service Binding OData V4 (Web API)

Questo e' il punto piu' critico. Per le app Fiori Elements su BTP ABAP, il manifest.json deve puntare a un Service Binding di tipo **OData V4 - Web API**, NON a un binding di tipo V4 - UI.

Tipo Binding	Uso	Gateway Registration	Funziona nel Launchpad?
OData V4 - UI	Solo preview da ADT	NON registra il service group	NO (errore 403)
OData V4 - Web API	URI nel manifest.json	Registra correttamente il service group	SI
OData V2 - UI	App con sap.ui.generic	Registra automaticamente	SI

Come creare il Service Binding V4 Web API in ADT:

- **File > New > Other > Service Binding**
- **Name:** ZSB_NOME_SERVIZIO_API (suffisso _API per convenzione)
- **Binding Type:** OData V4 - Web API
- **Service Definition:** selezionare la service definition esistente
- **Activate** (Ctrl+F3) e poi **Publish**

Il Service URL risultante sara':

```
/sap/opu/odata4/sap/zsb_nome_servizio_api/srvd_a2x/sap/zsd_nome_servizio/0001/
```

Questo URL va inserito nel manifest.json come dataSource.uri.

7. Deploy della BSP

7.1 Configurazione ui5-deploy.yaml

```
specVersion: "3.0"
metadata:
  name: nomeapp
type: application
builder:
  customTasks:
    - name: deploy-to-abap
      afterTask: generateCachebusterInfo
      configuration:
        target:
          url: https://SYSTEM_ID.abap.eu10.hana.ondemand.com
          destination: ABAP_Employee_RAP
        app:
          name: ZNOME_BSP
          package: ZPACKAGE
          transport: ""
```

7.2 Comandi di deploy

Eseguire in BAS (terminale):

```
# Build
npx ui5 build --clean-dest

# Deploy (primo deploy richiede --safe false)
npx fiori deploy --config ui5-deploy.yaml -- --safe false
```

Nota: Il primo deploy crea automaticamente la BSP, il LADI (Launchpad App Descriptor Item) e l'Inbound navigation.

8. Configurazione IAM e Business Catalog

Dopo il deploy, configurare le autorizzazioni in ADT:

8.1 Creare IAM App

- **File > New > Other > IAM App**
- **Name:** ZNOME_CONFIG_UI_EXT
- **Application Type:** External App
- **Fiori Launchpad App Descr Item ID:** inserire il LADI creato dal deploy (es. ZNOME_CONFIG_UI5R)
- Tab **Services:** aggiungere il servizio OData V4 (il binding Web API)
- **Publish Locally**

8.2 Assegnare al Business Catalog

Dalla IAM App, cliccare "**Assign the App to an existing Business Catalog**" e selezionare il catalogo appropriato.

8.3 Aggiungere IAM App auto-generata del binding API

Questo passo e' **essenziale per i tenant consumer**. Aprire il Business Catalog e aggiungere anche la IAM App auto-generata dal Service Binding V4 Web API (es. ZSB_NOME_API_EXT).

***Nota:** Senza questo passo, i tenant consumer riceveranno errore 403 'No authorization to access service group'.*

8.4 Pubblicare Business Catalog

- Aprire il Business Catalog in ADT
- **Publish Locally**
- Verificare che il Business Role associato mostri "Authorization is up-to-date"

9. Configurazione Tile nel Launchpad

Aggiungere il tile alla Page del Launchpad:

- Aprire il Launchpad > **Manage Launchpad Pages**
- Selezionare la Page appropriata
- Nella sezione desiderata, cercare l'app nel catalogo a destra
- Cliccare **Add** accanto all'app
- **Save**

Parametro	Esempio	Note
BSP App	ZDOC_CONFIG	Creata dal deploy
Tile title	Configurazione Conservazione	Titolo visibile nel Launchpad
Semantic Object	zdocconfig	Definito nel manifest.json (crossNavigation)
Action	display	Azione di navigazione
Icona	sap-icon://settings	Icona SAP standard

10. Communication Scenario e Arrangement

Il Communication Scenario autorizza l'accesso al servizio OData. Se esiste già un Communication Scenario per altre app dello stesso ecosistema, è possibile aggiungere il nuovo servizio a quello esistente.

Aggiungere il servizio al Communication Scenario:

- In ADT, aprire il Communication Scenario esistente
- Tab **Inbound** > **Add**
- Cercare l'Inbound Service del binding V4 Web API (es. ZSB_NOME_API_0001_G4BA)
- **Publish Locally**

La Communication Arrangement associata si aggiorna automaticamente con il nuovo servizio.

Configurazione Inbound:

Impostazione	Valore
Authentication Methods	Basic + X.509
Communication System	Es. ZDOC_BAS
User Name	Es. ZDOC_BAS_USER (utente tecnico)

11. Deploy su tenant consumer

Il deploy della BSP viene propagato automaticamente dal tenant provider (100) ai tenant consumer. Non serve un deploy separato. Tuttavia, le **autorizzazioni** richiedono configurazione specifica.

Checklist per tenant consumer

#	Azione	Dove
1	Deploy BSP dal tenant provider	BAS (npm run deploy) - solo una volta
2	IAM App auto-generata del binding API aggiunta al Business Catalog	ADT (Sezione 8.3)
3	Business Catalog pubblicato	ADT - Publish Locally
4	Servizio aggiunto al Communication Scenario	ADT (Sezione 10)
5	Verificare tile nel Launchpad del consumer	Browser

Nota: L'elemento chiave per i consumer e' il passo 2: la IAM App auto-generata del Service Binding V4 Web API deve essere nel Business Catalog. Senza questo, i consumer vedono il tile ma ricevono 403.

12. Multitenancy e isolamento dati

In SAP BTP ABAP Environment, l'isolamento dei dati tra tenant dipende dalla definizione delle tabelle database.

Tabella client-dependent (dati isolati per tenant):

```
define table ztabella {  
  key client      : abap.clnt not null; // Campo client  
  key campo_chiave : tipo not null;  
  campo_dato      : tipo;  
}
```

Delivery Class e isolamento:

Delivery Class	Tipo	Isolamento Tenant	Uso
#A	Application data	SI (con campo client)	Dati master gestiti dall'utente
#C	Customizing	SI (con campo client)	Dati di configurazione, trasportabili
#L	Temporary data	SI (con campo client)	Dati temporanei
#S	System table	NO (cross-client)	Dati di sistema condivisi

Requisiti per l'isolamento:

- **Campo client** : abap.clnt come prima chiave della tabella
- **Delivery Class #A o #C** (non #S)
- Il runtime ABAP filtra automaticamente per SY-MANDT
- In ABAP Cloud, CLIENT SPECIFIED non e' disponibile - l'isolamento e' garantito dal framework

Nota: Se una tabella ha delivery class #A ma NON ha il campo client, i dati sono condivisi tra tutti i tenant.

13. Lezioni apprese e best practice

Service Binding

Regola	Dettaglio
Usare V4 Web API per il manifest	Il binding V4 UI non registra il service group nel Gateway e causa 403
Suffisso _API per convenzione	Es. ZSB_NOME_SERVIZIO_API per il binding Web API
V4 UI solo per preview ADT	Il preview da ADT (click su entity > Preview) usa il binding UI

Frontend UI5

Regola	Dettaglio
Bootstrap locale, non CDN	Usare resources/sap-ui-core.js - il CDN causa pagina bianca in BAS
Versione semver	Usare 1.120.0 (non 1.120) nei file yaml
Testi diretti nelle annotations	Il binding {i18n>key} non funziona nei file XML esterni
Component.js obbligatorio	Deve estendere sap/fe/core/AppComponent
Primo deploy con --safe false	Il flag e' necessario per la prima creazione della BSP

Autorizzazioni e tenant

Regola	Dettaglio
IAM App auto-generata nel BC	Aggiungere la IAM App del binding API al Business Catalog per i consumer
Communication Scenario	Aggiungere il servizio API all'Inbound del Communication Scenario
Campo client per isolamento	Senza il campo abap.cnt nella tabella, i dati sono condivisi tra tenant
Pubblicare sempre	Dopo ogni modifica: Publish Locally su IAM App, BC e Communication Scenario